QJL - 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead - Paper Notes

QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead

#Research

Source paper: AAAI 2025 paper qjl-34773.pdf
Version: published version (AAAI 2025, issued April 11, 2025)
Category: AAAI / KV Cache Quantization / Johnson-Lindenstrauss / Sketching
Authors: Amir Zandieh, Majid Daliri, Insu Han
Extended version: arXiv:2406.03482

Quick Orientation

- Core problem: existing key-cache quantizers usually spend extra memory on per-block quantization constants, which undermines their claimed bit savings.
- Core idea: apply a Johnson-Lindenstrauss transform to each key vector, keep only the sign bits, and recover attention scores with an asymmetric estimator that uses an unquantized query vector.
- Why it matters: the method gets effectively zero quantization-constant overhead for keys, supports 3-bit KV-cache settings, and keeps downstream quality close to 16-bit baselines.
- Main limitations: the strongest theorem needs bounded query/key norms and only covers the key-side estimator; the value cache is still quantized with a more standard token-wise method.

Abstract + Executive Summary Notes

- QJL is really two things:
  - a 1-bit quantized sketch `H_S(k) = sign(Sk)`,
  - an asymmetric inner-product estimator `Prod_QJL(q, k)` that multiplies the unquantized projected query with the sign-only key sketch.
- The paper's central theoretical surprise is that quantizing only one side to a single sign bit still preserves unbiased inner-product estimation with low distortion.
- The motivating use case is attention decoding: if the key cache can be stored as sign bits plus norms, memory traffic during decoding drops sharply.

- The main empirical claim is that 3-bit QJL can reduce KV-cache memory by over 5x with little or no accuracy loss, while also improving decoding speed in long-context settings.
- A second practical claim is compatibility: the kernels support grouped-query attention and BF16, which lets the method run on models where some earlier baselines do not.

Section 1: Attention Decoding Setup

- The paper begins by restating attention decoding as repeated inner products between the current query vector and all cached key vectors.
- That makes the key cache the right target for a sketch-based method: if you can preserve those inner products, you preserve the attention scores.
- The cost bottleneck is not only storage but also bandwidth. Every decoding step pulls the cache from GPU memory, so smaller key representations directly help runtime.

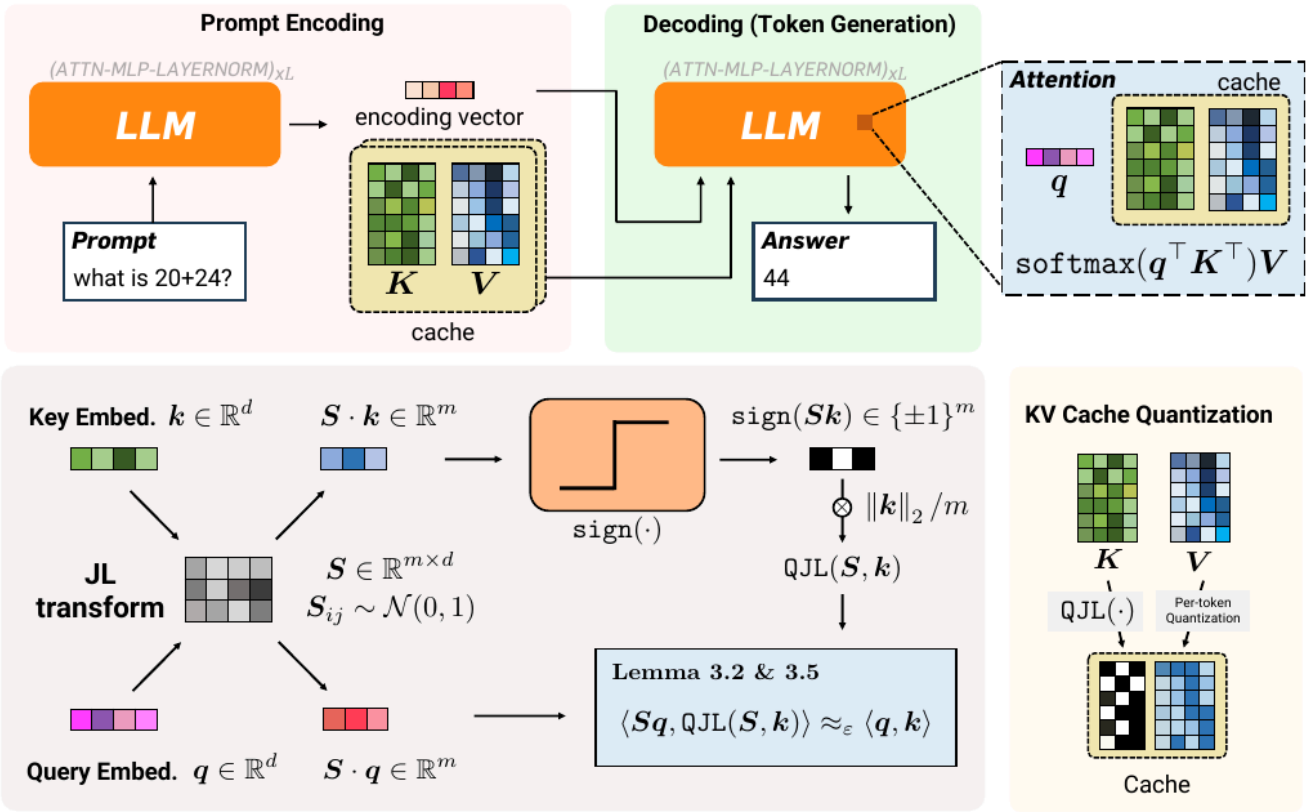


Figure 1: Overview of the KV cache quantization via Quantized JL (QJL) transform

Figure 1. QJL stores sign-only JL sketches of key vectors plus norms, then combines them with an unquantized projected query at decode time.

Section 2: Quantized Johnson-Lindenstrauss

- The method defines `H_S(k) = sign(Sk)` where `S` is a Gaussian JL matrix.
- The key estimator is asymmetric:
  - the key is quantized to sign bits,
  - the query is projected but not quantized.
- This matters because quantizing both sides would estimate angle well but would not give an unbiased inner-product estimator after the cosine/nonlinearity.
- Lemma 2 proves the estimator is unbiased.
- Lemma 3 proves a low-distortion bound for the estimator, with sketch dimension scaling like `epsilon^-2 log(1/delta)`.
- Theorem 4 then lifts this to attention decoding:
  - if key and query norms are bounded by `r`,
  - and `m >= 2 r^2 epsilon^-2 log n`,
  - then all estimated attention scores are within about `1 +/- 3 epsilon` multiplicative distortion with high probability.

Section 3: KV Cache Quantization Design

Key Cache

- Algorithm 1 stores, per token:
  - the sign sketch `k_tilde`,
  - the original key norm `nu`.
- The paper highlights an important asymptotic property: the number of bits per key depends logarithmically on context length and does not scale with the original embedding dimension in the same way dense storage does.
- This is the "zero overhead" part of the title: there are no per-group scale and zero-point tensors for the key cache.

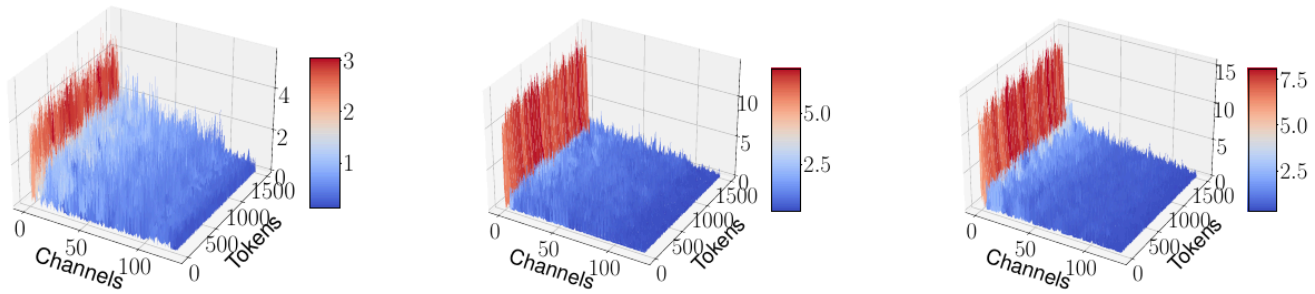
Value Cache

- The value side is intentionally conventional.
- The paper uses standard token-wise quantization for values, arguing that earlier work already showed this part is easy and low-risk.
- So the novelty is specifically on the key side, where attention-score sensitivity makes the problem harder.

Section 4: Practical Considerations

Outliers

- The authors observe that outliers are not uniform across the model:
  - early layers have few strong outliers,
  - deeper layers show a few fixed channels with much larger magnitudes.
- Since the distortion bounds scale with embedding norms, these channels matter disproportionately.
- Their fix is mixed precision by channel:
  - identify the outlier channels,
  - quantize them with a separate QJL instance using more bits,
  - quantize the rest more aggressively.



(a) Layer 0, Head 0 (b) Layer 15, Head 0 (c) Layer 31, Head 0

Figure 2: The magnitude of key cache entries for different layers of the Llama-2 model, for an example prompt, reveals notable patterns. Channels are sorted by their average magnitudes. In initial layers, no significant outliers are observed. However, in the deeper layers, few channels (approximately four) exhibit visibly larger magnitudes (outliers), highlighting the importance of addressing these outliers to improve quantization accuracy for the key cache.

Figure 2. Large outlier channels are mostly a deeper-layer phenomenon, which motivates treating a small subset of channels differently.

Orthogonalized JL

- The paper also reports an empirical improvement from orthogonalizing the rows of the JL matrix.
- That does not change the high-level method, but it is a useful practical detail because it suggests the sketch matrix itself is worth engineering rather than treating as a throwaway random draw.

Section 5: Experiments

Ablation on Distortion

- Figure 3 measures the mean relative distortion on attention scores versus bits per token/channel.

- The key qualitative finding is that the first layer is much harder to quantize than the deeper layers.
- The curve also supports the expected `m ~ 1/epsilon^2` relationship from the theory.

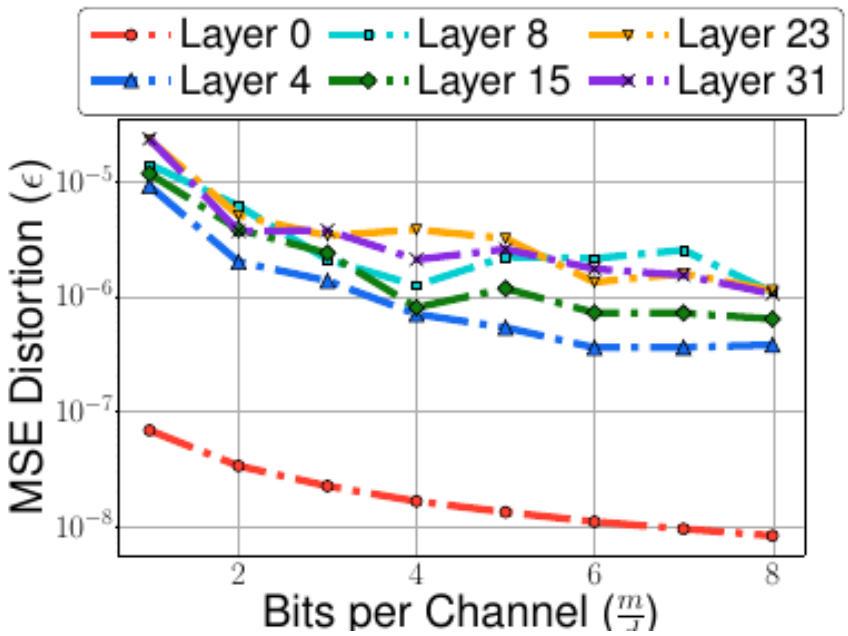


Figure 3. The first layer is the hardest to quantize, while later layers get low distortion with relatively modest sketch sizes.

LongBench and Long-Context Results

- On the longchat-7b-v1.5-32k setup, QJL is compared against KIVI and KVQuant at matched or near-matched effective bit budgets.
- QJL is not uniformly best on every task, but it stays competitive at very low bit-width and often beats KIVI on QA-heavy metrics like HotpotQA and Qasper.
- On Llama-3.1-8B-Instruct, the reported averages are:
  - BF16 baseline: 53.47
  - QJL 3-bit: 52.48
  - QJL 4-bit: 52.82
- So the degradation is small enough that the compression looks operationally viable, especially given the memory savings.

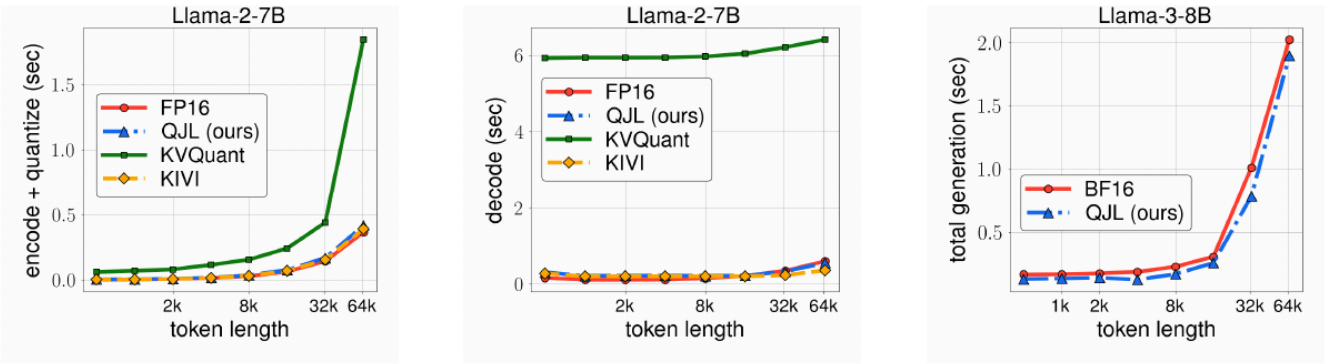
Standard-Length Accuracy

- The paper also checks LM-eval tasks outside the long-context regime.

- On Llama-2-7B, 3-bit QJL is effectively tied with the 16-bit baseline across Lambada, HellaSwag, PIQA, MathQA, and MMLU.
- On Llama-3-8B, 3-bit QJL is similarly near-identical to BF16.
- This matters because it suggests the sketch is not only preserving long-context behavior; it is also not introducing broad quality regressions on more ordinary evaluation sets.

Runtime and Memory

- Figure 4 compares prompt encoding, KV-cache quantization, and decoding time.
- The most useful summary:
  - QJL and KIVI both decode faster than the exact baseline at 3 bits,
  - KVQuant is much slower because of heavier preprocessing,
  - QJL is the only method in the comparison that can quantize Llama-3 in this setup.



(a) Prompt encoding (Llama2) (b) Token generation (Llama2) (c) Encode and generate (Llama3)

Figure 4: Wall-clock time to encode a prompt and quantize the KV cache (left), generate 128 tokens for llama2 model (middle), and generate 64 tokens for llama3 model (right) using different quantization methods in a single attention layer. The input sequence length varies from 1k to 64k. Both KIVI and QJL (ours) with 3 bits per FPN show faster decoding time than the baseline. However, KVQuant is significantly slower during both quantizing and decoding phases. QJL is the only method that can quantize Llama3, as our kernels support grouped query attention and BF16 data type. We observe the same speed for Llama3 as the exact method for generation. Our memory usage is at least 5-fold less than the exact method and can support all data types.

Models	Methods	Bits	Datasets from LM-eval (Gao et al. 2023)				
			Lambada-OpenAI	HellaSwag	PIQA	MathQA	MMLU

Figure 4. QJL reduces decoding cost relative to the exact baseline while avoiding the heavy preprocessing cost seen in KVQuant.

- Figure 5 shows peak memory for prompt encoding plus generation on Llama-2.
- The paper says KV-cache memory drops by at least 5x relative to the exact method, and even total peak memory drops by more than 2x once model weights are included.

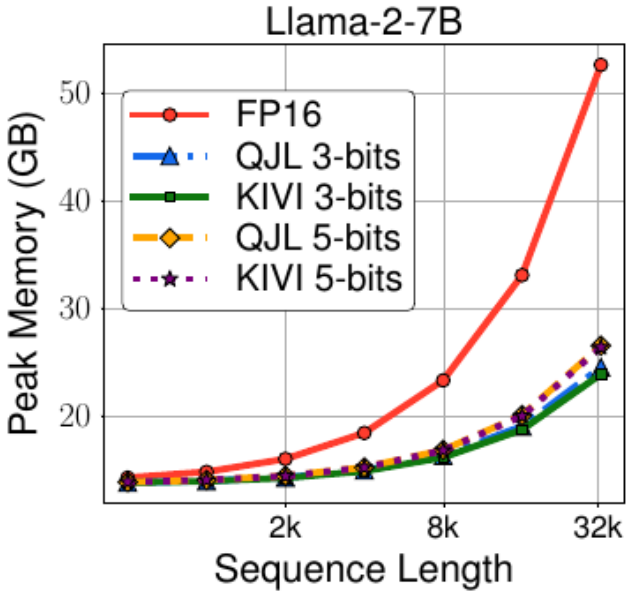


Figure 5. Peak memory falls sharply under 3-bit and 5-bit QJL, showing that the cache savings remain visible even after accounting for model-weight memory.

Conclusion Notes

- The paper's most important contribution is the asymmetric estimator, not merely the sign sketch itself.
- That estimator is what makes a 1-bit key representation viable for attention-score estimation instead of just for crude similarity hashing.
- Practically, QJL is strongest where bandwidth matters: long-context decoding, large KV caches, and settings where per-group quantization overhead is expensive.
- Conceptually, it is a sketching paper disguised as a systems paper, and that is why the theoretical guarantees are unusually crisp for this part of the LLM compression literature.

Appendix Highlights Worth Remembering

- The published AAAI paper repeatedly points readers to the extended arXiv version for full proofs, so the workshop-level intuition and the longer derivations are split across two artifacts.
- The value-cache path is intentionally conservative, which keeps the contribution focused on the key-side estimator rather than trying to redesign the entire KV stack at once.
- Figures detected in the main paper run from Figure 1 through Figure 5, and all five are embedded above.

Study Checklist

- Why does QJL quantize only the key-side sketch to sign bits instead of quantizing both query and key?
- What does "zero overhead" mean in this paper, and what overhead still remains elsewhere in the KV pipeline?
- Why do outlier channels matter so much for the performance of QJL?
- What does Theorem 4 actually guarantee about attention scores?
- How close does QJL stay to the BF16/FP16 baselines on the reported long-context and LM-eval tables?
- Why is QJL better interpreted as a sketching method than as an ordinary scalar quantizer?

High-Yield Concept List

QJL
Johnson-Lindenstrauss Transform
Asymmetric Inner-Product Estimator
KV Cache Quantization
Attention Decoding
Outlier Channels
Sketching

Study Answer Key

1) Why does QJL quantize only the key-side sketch to sign bits instead of quantizing both query and key?

Because the paper wants an unbiased estimator for the inner product, not just a rough angle estimate. If both sides were quantized to signs, the result would behave like a similarity-hashing estimator for angle and would not remain an unbiased inner-product estimator after the needed rescaling. The asymmetry is what preserves unbiasedness.

2) What does "zero overhead" mean in this paper, and what overhead still remains elsewhere in the KV pipeline?

It means the key cache no longer needs per-block scales or zero points stored in full precision. The keys are stored as sign sketches plus norms. Overhead still remains in

the broader KV pipeline because value vectors are still quantized with a more standard token-wise method and the system still stores the per-token key norms.

3) Why do outlier channels matter so much for the performance of QJL?

Because the estimator's distortion scales with the norms of the embeddings. A few very large channels can dominate those norms, so quantizing them too aggressively hurts attention-score accuracy disproportionately. Treating them separately is a direct way to reduce the worst distortion source.

4) What does Theorem 4 actually guarantee about attention scores?

It says that if the sketch dimension is large enough relative to the norm bound and context length, then all estimated attention scores are simultaneously close to the exact scores with high probability, specifically within roughly a `1 +/- 3 epsilon` multiplicative factor.

5) How close does QJL stay to the BF16/FP16 baselines on the reported long-context and LM-eval tables?

Quite close. On the Llama-3.1-8B-Instruct long-context table, the 3-bit and 4-bit QJL averages are only modestly below the BF16 baseline. On LM-eval, 3-bit QJL is nearly indistinguishable from the full-precision baselines on both Llama-2-7B and Llama-3-8B.

6) Why is QJL better interpreted as a sketching method than as an ordinary scalar quantizer?

Because it works by projecting vectors into a random low-dimensional space and then estimating inner products from the sketch, not by independently rounding each original coordinate. Its analysis, guarantees, and practical value all come from randomized sketching theory rather than from classic per-coordinate scalar quantization alone.